

Ejemplo usando Planificador STRIPS

Dominio de Aplicación: Delivery Robot

En este dominio de aplicación existe un agente (robot) capaz de llevar cajas de una habitación a otra. Las habitaciones están separadas entre sí por una puerta que puede estar cerrada con llave. El robot sólo se puede mover entre habitaciones adyacentes y puede recolectar todo lo que esté en la misma habitación en la que él se encuentra. El robot puede poseer varias llaves, pero solo puede cargar una caja a la vez.

Para definir un escenario dentro de este dominio de aplicación pueden utilizarse las siguientes **relaciones estáticas**:

- llave(L): Representa que L es una llave.
- caja(C): Representa que C es una caja.
- puerta(HA, HB, P): Representa que las habitaciones HA, y HB están conectadas por la puerta P.
- abre(L, P): Representa que la llave L abre la puerta P. Por simplicidad, vamos a asumir que todas las puertas necesitan una llave para abrirse.

Además, se cuenta con la siguiente **relación derivada (estática)**:

- adyacentes(HA,HB, P) $\leftarrow$ puerta(HA, HB, P).
- adyacentes(HB,HA, P) $\leftarrow$ puerta(HA, HB, P).

Además, tendremos las siguientes **relaciones dinámicas**:

- esta_en(Ent,Hab): Representa que la entidad Ent está en la habitación Hab. La entidad Ent puede ser el robot, una llave, o una caja; y Hab es una de las habitaciones del escenario.
- posee(Obj): Representa que el robot posee el objeto Obj, que puede ser una llave o una caja.
- sin_caja: Representa que el robot actualmente no está cargando una caja.

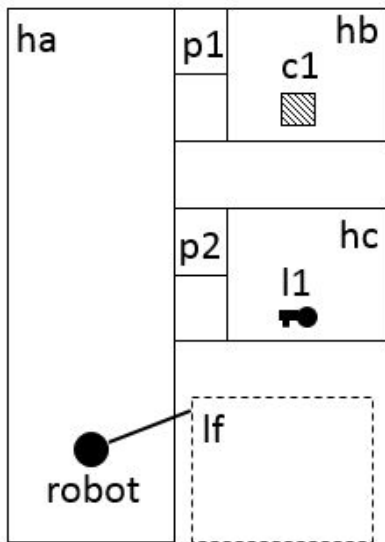
Dominio de Aplicación: Delivery Robot

La siguiente tabla describe los operadores que el robot de delivery podría aplicar para alcanzar sus metas.

Operador	Pre-condiciones	Efectos	
		Add-list	Del-list
ir_a(HB)	{adyacentes(HA,HB,P), abre(L,P), posee(L), esta_en(robot,HA)}	{esta_en(robot,HB)}	{esta_en(robot,HA)}
levantar_llave(L)	{llave(L), esta_en(L,H), esta_en(robot,H)}	{posee(L)}	{esta_en(L,H)}
levantar_caja(C)	{caja(C), esta_en(C,H), esta_en(robot,H), sin_caja}	{posee(C)}	{esta_en(C,H), sin_caja}
soltar_caja(C)	{caja(C), posee(C), esta_en(robot,H)}	{esta_en(C,H), sin_caja}	{posee(C)}

Problema de Planificación

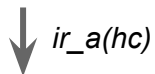
Vamos a utilizar el planificador STRIPS para resolver un problema de planificación donde el estado inicial E0 está representado por la figura que se muestra debajo. La meta a alcanzar es $M = \{ \text{esta_en}(c1, hc), \text{esta_en}(\text{robot}, ha) \}$.



```
E0 = { esta_en(robot, ha), esta_en(c1, hb),  
      esta_en(l1, hc) .  
      posee(lf) .  
      puerta(ha, hb, p1), puerta(ha, hc, p2) .  
      abre(lf, p2), abre(l1, p1),  
      llave(lf), llave(l1), caja(c1) .  
      sin_caja.  
}
```

Traza

```
E0 = { esta_en(robot,ha), esta_en(c1,hb),
      esta_en(l1,hc).
      posee(lf).
      puerta(ha,hb,p1), puerta(ha,hc,p2).
      abre(lf,p2), abre(l1,p1),
      llave(lf), llave(l1), caja(c1).
      sin_caja.
    }
```



ir_a(hc)

```
E1 = { esta_en(robot,hc), esta_en(c1,hb),
      esta_en(l1,hc).
      posee(lf).
      puerta(ha,hb,p1), puerta(ha,hc,p2).
      abre(lf,p2), abre(l1,p1),
      llave(lf), llave(l1), caja(c1).
      sin_caja.
    }
```

Recordar que también contamos con la relación estática derivada *adyacentes*.

	alcanzar		ejecutar
1	esta_en(c1,hc)		soltar_caja(c1)
2	esta_en(robot, ha)		
1.1	caja(c1)	E0	levantar_caja(c1)
1.2	posee(c1)		
1.3	esta_en(robot, hc)		
1.2.1	caja(c1)	E0	
1.2.2	esta_en(c1, hb)	E0	ir_a(hb)
1.2.3	esta_en(robot, hb)		
1.2.4	sin_caja		
1.2.3.1	adyacentes(ha, hb, p1)	E0	
1.2.3.2	abre(l1, p1)	E0	levantar_llave(l1)
1.2.3.3	posee(l1)		
1.2.3.4	esta_en(robot, ha)		
1.2.3.3.1	llave(l1)	E0	
1.2.3.3.2	esta_en(l1, hc)	E0	ir_a(hc)
1.2.3.3.3	esta_en(robot, hc)	E1	
1.2.3.3.3.1	adyacentes(ha, hc, p2)	E0	
1.2.3.3.3.2	abre(lf, p2)	E0	
1.2.3.3.3.3	posee(lf)	E0	
1.2.3.3.3.4	esta_en(robot, ha)	E0	

Traza

```
E1 = { esta_en(robot,hc), esta_en(c1,hb),  
      esta_en(l1,hc).  
      posee(lf).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      sin_caja.  
}
```



levantar_llave(l1)

```
E2 = { esta_en(robot,hc), esta_en(c1,hb).  
      posee(lf). posee(l1).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      sin_caja.  
}
```

	alcanzar		ejecutar
1 2	esta_en(c1,hc) esta_en(robot, ha)		soltar_caja(c1)
1.1 1.2 1.3	caja(c1) posee(c1) esta_en(robot, hc)	E0	levantar_caja(c1)
1.2.1 1.2.2 1.2.3 1.2.4	caja(c1) esta_en(c1, hb) esta_en(robot, hb) sin_caja	E0 E0	ir_a(hb)
1.2.3.1 1.2.3.2 1.2.3.3 1.2.3.4	adyacentes(ha, hb, p1) abre(l1, p1) posee(l1) esta_en(robot, ha)	E0 E0 E2	levantar_llave(l1)
1.2.3.3.1 1.2.3.3.2 1.2.3.3.3	llave(l1) esta_en(l1, hc) esta_en(robot, hc)	E0 E0 E1	ir_a(hc)
...			

Por cuestiones de espacio, vamos a poner puntos suspensivos para las precondiciones de acciones que ya hayamos proyectado (en este caso, *ir_a(hc)*)

Traza

```
E2 = { esta_en(robot, hc), esta_en(c1, hb).
      posee(lf). posee(l1).
      puerta(ha, hb, p1), puerta(ha, hc, p2).
      abre(lf, p2), abre(l1, p1),
      llave(lf), llave(l1), caja(c1).
      sin_caja.
    }
```

↓ *ir_a(ha)*

```
E3 = { esta_en(robot, ha), esta_en(c1, hb).
      posee(lf). posee(l1).
      puerta(ha, hb, p1), puerta(ha, hc, p2).
      abre(lf, p2), abre(l1, p1),
      llave(lf), llave(l1), caja(c1).
      sin_caja.
    }
```

	alcanzar		ejecutar
1 2	esta_en(c1, hc) esta_en(robot, ha)		soltar_caja(c1)
1.1 1.2 1.3	caja(c1) posee(c1) esta_en(robot, hc)	E0	levantar_caja(c1)
1.2.1 1.2.2 1.2.3 1.2.4	caja(c1) esta_en(c1, hb) esta_en(robot, hb) sin_caja	E0 E0	ir_a(hb)
1.2.3.1 1.2.3.2 1.2.3.3 1.2.3.4	adyacentes(ha, hb, p1) abre(l1, p1) posee(l1) esta_en(robot, ha)	E0 E0 E2 E3	levantar_llave(l1) ir_a(ha)
...			
1.2.3.4.1 1.2.3.4.2 1.2.3.4.3 1.2.3.4.4	adyacentes(hc, ha, p2) abre(lf, p2) posee(lf) esta_en(robot, hc)	E2 E2 E2 E2	

Traza

```
E3 = { esta_en(robot,ha), esta_en(c1,hb).  
      posee(lf). posee(l1).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      sin_caja.  
}
```

↓ **ir_a(hb)**

```
E4 = { esta_en(robot,hb), esta_en(c1,hb).  
      posee(lf). posee(l1).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      sin_caja.  
}
```

	alcanzar		ejecutar
1 2	esta_en(c1,hc) esta_en(robot, ha)		soltar_caja(c1)
1.1 1.2 1.3	caja(c1) posee(c1) esta_en(robot, hc)	E0	levantar_caja(c1)
1.2.1 1.2.2 1.2.3 1.2.4	caja(c1) esta_en(c1, hb) esta_en(robot, hb) sin_caja	E0 E0 E4	ir_a(hb)
1.2.3.1 1.2.3.2 1.2.3.3 1.2.3.4	adyacentes(ha, hb, p1) abre(l1, p1) posee(l1) esta_en(robot, ha)	E0 E0 E2 E3	levantar_llave(l1) ir_a(ha)
...			

Traza

```
E4 = { esta_en(robot,hb), esta_en(c1,hb).  
      posee(lf). posee(l1).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      sin_caja.  
}
```

↓ *levantar_caja(c1)*

```
E5 = { esta_en(robot,hb).  
      posee(lf). posee(l1).  
      puerta(ha,hb,p1), puerta(ha,hc,p2).  
      abre(lf,p2), abre(l1,p1),  
      llave(lf), llave(l1), caja(c1).  
      posee(c1).  
}
```

	alcanzar		ejecutar
1 2	esta_en(c1,hc) esta_en(robot, ha)		soltar_caja(c1)
1.1 1.2 1.3	caja(c1) posee(c1) esta_en(robot, hc)	E0 E5	levantar_caja(c1)
1.2.1 1.2.2 1.2.3 1.2.4	caja(c1) esta_en(c1, hb) esta_en(robot, hb) sin_caja	E0 E0 E4 E4	ir_a(hb)
1.2.3.1 1.2.3.2 1.2.3.3 1.2.3.4	adyacentes(ha, hb, p1) abre(l1, p1) posee(l1) esta_en(robot, ha)	E0 E0 E2 E3	levantar_llave(l1) ir_a(ha)
...			

Traza

```
E5 = { esta_en(robot,hb).  
       posee(lf). posee(l1).  
       puerta(ha,hb,p1), puerta(ha,hc,p2).  
       abre(lf,p2), abre(l1,p1),  
       llave(lf), llave(l1), caja(c1).  
       posee(c1).  
 }
```

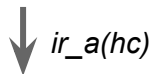
↓ *ir_a(ha)*

```
E6 = { esta_en(robot,ha).  
       posee(lf). posee(l1).  
       puerta(ha,hb,p1), puerta(ha,hc,p2).  
       abre(lf,p2), abre(l1,p1),  
       llave(lf), llave(l1), caja(c1).  
       posee(c1).  
 }
```

alcanzar			ejecutar
1	esta_en(c1,hc)		soltar_caja(c1)
2	esta_en(robot, ha)		
1.1	caja(c1)	E0	levantar_caja(c1) ir_a(hc)
1.2	posee(c1)	E5	
1.3	esta_en(robot, hc)		
...			
1.3.1	adyacentes(ha, hc, p2)	E5	ir_a(ha)
1.3.2	abre(lf, p2)	E5	
1.3.3	posee(lf)	E5	
1.3.4	esta_en(robot, ha)	E6	
1.3.4.1	adyacentes(hb, ha, p1)	E5	
1.3.4.2	abre(l1, p1)	E5	
1.3.4.3	posee(l1)	E5	
1.3.4.4	esta_en(robot, hb)	E5	

Traza

```
E6 = { esta_en(robot,ha).  
       posee(lf). posee(l1).  
       puerta(ha,hb,p1), puerta(ha,hc,p2).  
       abre(lf,p2), abre(l1,p1),  
       llave(lf), llave(l1), caja(c1).  
       posee(c1).  
 }
```



```
E7 = { esta_en(robot,hc).  
       posee(lf). posee(l1).  
       puerta(ha,hb,p1), puerta(ha,hc,p2).  
       abre(lf,p2), abre(l1,p1),  
       llave(lf), llave(l1), caja(c1).  
       posee(c1).  
 }
```

alcanzar			ejecutar
1	esta_en(c1,hc)		soltar_caja(c1)
2	esta_en(robot, ha)		
1.1	caja(c1)	E0	levantar_caja(c1) ir_a(hc)
1.2	posee(c1)	E5	
1.3	esta_en(robot, hc)	E7	
...			
1.3.1	adyacentes(ha, hc, p2)	E5	ir_a(ha)
1.3.2	abre(lf, p2)	E5	
1.3.3	posee(lf)	E5	
1.3.4	esta_en(robot, ha)	E6	
1.3.4.1	adyacentes(hb, ha, p1)	E5	
1.3.4.2	abre(l1, p1)	E5	
1.3.4.3	posee(l1)	E5	
1.3.4.4	esta_en(robot, hb)	E5	

Traza

```
E7 = { esta_en(robot, hc).  
       posee(lf). posee(l1).  
       puerta(ha, hb, p1), puerta(ha, hc, p2).  
       abre(lf, p2), abre(l1, p1),  
       llave(lf), llave(l1), caja(c1).  
       posee(c1).  
 }
```

↓ *soltar_caja(c1)*

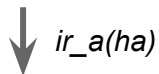
```
E8 = { esta_en(robot, hc).  
       posee(lf). posee(l1).  
       puerta(ha, hb, p1), puerta(ha, hc, p2).  
       abre(lf, p2), abre(l1, p1),  
       llave(lf), llave(l1), caja(c1).  
       esta_en(c1, hc).  
       sin_caja.  
 }
```

	alcanzar		ejecutar
1 2	esta_en(c1, hc) esta_en(robot, ha)	E8	soltar_caja(c1)
1.1 1.2 1.3	caja(c1) posee(c1) esta_en(robot, hc)	E0 E5 E7	levantar_caja(c1) ir_a(hc)
...			

Cumplimos con la primera de las submetas,
ahora debemos cumplir la segunda...

Traza

```
E8 = { esta_en(robot, hc).  
       posee(lf). posee(l1).  
       puerta(ha, hb, p1), puerta(ha, hc, p2).  
       abre(lf, p2), abre(l1, p1),  
       llave(lf), llave(l1), caja(c1).  
       esta_en(c1, hc).  
       sin_caja.  
 }
```



```
E9 = { esta_en(robot, ha).  
       posee(lf). posee(l1).  
       puerta(ha, hb, p1), puerta(ha, hc, p2).  
       abre(lf, p2), abre(l1, p1),  
       llave(lf), llave(l1), caja(c1).  
       esta_en(c1, hc).  
       sin_caja.  
 }
```

alcanzar			ejecutar
1	esta_en(c1, hc)	E8	soltar_caja(c1)
2	esta_en(robot, ha)	E9	ir_a(ha)
...			
2.1	adyacentes(hc, ha, p2)	E8	
2.2	abre(lf, p2)	E8	
2.3	posee(lf)	E8	
2.4	esta_en(robot, hc)	E8	

Todas las metas fueron alcanzadas, por lo que al recopilar todas las acciones obtenemos una solución (plan).

Solución: [ir_a(hc), levantar_llave(l1), ir_a(ha), ir_a(hb),
levantar_caja(c1), ir_a(ha), ir_a(hc), soltar_caja(c1), ir_a(ha)]

Para reflexionar:

¿Qué resultado alcanzaría el algoritmo si tomamos las metas en el orden inverso, es decir, [esta_en(robot, ha), esta_en(c1, hc)]?
¿Encontraría una solución al problema de planificación?